

گزارش فاز دوم پروژه  
درس هوش مصنوعی و سیستم‌های خبره

اعضای گروه:

زهرا علی‌محمدی

فاطمه سمساری

نگار شفیع‌ی

## (۱) اطلاعات پایه پروژه:

### ۱،۱- معرفی پروژه و مسئله

این پروژه با هدف تشخیص نفوذ (Intrusion Detection) طراحی شده است. داده‌ها شامل ویژگی‌های مربوط به ترافیک شبکه و رفتار کاربر در نشست‌های دسترسی هستند و هدف، تشخیص فعالیت‌های مخرب (مانند تلاش‌های ناموفق متعدد، دسترسی در زمان غیرمعمول، یا الگوهای غیرعادی ترافیک) در قالب یک مسئله طبقه‌بندی دودویی است. (مسیر دو - یادگیری ماشین کلاسیک صنعتی)

### ۱،۲- معرفی دیتاست

دیتاست مورد استفاده «Cybersecurity Intrusion Detection Dataset» است که هر ردیف آن نمایانگر یک session/access event بوده و ترکیبی از ویژگی‌های فنی شبکه و الگوهای رفتاری کاربر را ثبت می‌کند. این ترکیب باعث می‌شود مدل بتواند هم نشانه‌های سطح پایین شبکه و هم سرخ‌های رفتاری مرتبط با نفوذ را یاد بگیرد.

### مشخصات کلی دیتاست:

- مسیر فایل خام: data/final/cybersecurity\_intrusion\_data\_v1.csv
- تعداد نمونه‌ها: 9537
- تعداد ستون‌ها: 11
- برچسب هدف: attack\_detected

### ۱،۳- ویژگی‌ها (Features)

ویژگی‌ها در دو گروه اصلی قرار می‌گیرند:

الف) ویژگی‌های مبتنی بر شبکه

- network\_packet\_size: اندازه بسته‌های شبکه (بایت)
- protocol\_type: پروتکل ارتباطی (TCP/UDP/ICMP)
- encryption\_used: رمزنگاری (AES/DES/None)

ب) ویژگی‌های رفتاری کاربر

- login\_attempts: تعداد تلاش‌های ورود

- **failed\_logins:** تعداد ورود ناموفق
- **session\_duration:** مدت زمان نشست (ثانیه)
- **unusual\_time\_access:** دسترسی در زمان غیرمعمول (0/1)
- **ip\_reputation\_score:** امتیاز شهرت IP (۰ تا ۱)
- **browser\_type:** نوع مرورگر (Chrome/Firefox/Edge/Safari/Unknown)

نکته: طبق داده‌دیکشنری، برخی ستون‌ها ماهیت **categorical** (مثل protocol\_type) و برخی **عددی/باینری** دارند که در مرحله پیش‌پردازش برای مدل‌سازی باید به شکل مناسب تبدیل شوند.

#### ۱،۴ - متغیر هدف (Target)

- **attack\_detected = 1** --> حمله / فعالیت مخرب
- **attack\_detected = 0** --> رفتار عادی / benign

#### ۱،۵ - محیط اجرا و وابستگی‌ها

برای اجرای پروژه، کتابخانه‌های کلیدی زیر استفاده شده‌اند:

- تحلیل داده: numpy, pandas
- مصورسازی: matplotlib, seaborn
- مدل‌سازی: scikit-learn و همچنین xgboost
- ذخیره/بارگذاری مدل: joblib
- محیط نوت‌بوک: jupyter, ipykernel, notebook/jupyterlab

## ۲) کدها و نوت‌بوک‌های فاز دوم (Training + Evaluation)

در فاز دوم، فرآیند مدل‌سازی بر پایه‌ی **XGBoost (XGBClassifier)** پیاده‌سازی شده و برای ارزیابی، هم متریک‌های استاندارد طبقه‌بندی محاسبه شده‌اند و هم نمودارهای کلیدی (Confusion Matrix، ROC و منحنی logloss آموزش) تولید و ذخیره می‌شوند.

### ۲,۱- ساختار کلی ماژول‌ها و فایل‌ها

فایل‌های اصلی فاز دوم شامل موارد زیر است:

- محاسبه متریک‌ها: تابع `compute_classification_metrics` برای محاسبه‌ی Confusion Matrix و Accuracy/Precision/Recall/F1/ROC-AUC
- رسم نمودارها: توابع رسم Confusion Matrix، ROC Curve و Training Logloss Curve
- **Feature Engineering**: ساخت ۴ ویژگی امنیتی جدید بر اساس تعامل بین رفتار ورود، زمان دسترسی، شدت ترافیک و ریسک IP
- **Training نسخه پایه (xgb\_v0)**: آموزش XGB روی داده‌های پیش‌پردازش‌شده و ارزیابی روی Validation و Test با آستانه پیش‌فرض مدل
- **Training نسخه بهبود یافته (xgb\_v1)**: آموزش مدل + tuned افزودن + Feature Engineering کاهش آستانه تصمیم روی Test برای افزایش Recall

### ۲,۲- محاسبه متریک‌ها (Evaluation Metrics)

برای ارزیابی عملکرد مدل، یک تابع واحد طراحی شده که متریک‌های زیر را برمی‌گرداند:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

- Confusion Matrix (به صورت لیست دوبعدی برای ذخیره در JSON)

این کار باعث می‌شود ارزیابی روی مجموعه‌های Validation و Test به شکل یکسان و قابل مقایسه انجام شود.

### ۲,۳- تولید نمودارها (Plots)

برای مستندسازی خروجی‌ها و ارائه‌ی بصری عملکرد مدل، نمودارهای زیر تولید و در مسیر خروجی ذخیره می‌شوند:

#### ۱. Confusion Matrix

نمایش تعداد True/False Positive و True/False Negative برای دو کلاس Normal و Attack.

#### ۲. ROC Curve + AUC

محاسبه‌ی منحنی ROC با roc\_curve و نمایش مقدار AUC روی نمودار.

#### ۳. Training Logloss Curve

رسم logloss آموزش و اعتبارسنجی در طول iteration ها (در صورت موجود بودن evals\_result). این نمودار برای بررسی Overfitting/Underfitting مفید است.

### ۲,۴- Feature Engineering (افزودن ویژگی‌های امنیتی)

در نسخه‌ی بهبود یافته، ۴ ویژگی جدید ساخته شده که هدفشان تقویت سیگنال‌های مرتبط با رفتار مشکوک است:

- login\_pressure = login\_attempts × failed\_logins  
فشار تلاش‌های ورود + تعداد خطا (نشانه‌ی brute force)
- time\_fail\_risk = unusual\_time\_access × failed\_logins  
خطاهای ورود در زمان غیرمعمول (الگوی مشکوک‌تر)
- traffic\_intensity = network\_packet\_size × session\_duration  
شدت ترافیک در یک نشست (ترکیب حجم و مدت)
- ip\_attack\_risk = ip\_reputation\_score × failed\_logins  
ترکیب اعتبار IP و ورود ناموفق (ریسک حمله بالاتر)

این ویژگی‌ها هم روی داده‌ی Train و هم Test اعمال می‌شوند تا سازگاری ورودی مدل حفظ شود.

### ۲,۵- آموزش مدل نسخه پایه xgb\_v0 :

در فایل آموزش نسخه‌ی پایه:

- داده‌های پیش‌پردازش‌شده‌ی Train/Test بارگذاری می‌شوند (با وجود ستون هدف attack\_detected).
- داده‌ی Train با Stratify به Train/Validation تقسیم می‌شود. (VAL\_SIZE = 0.2)
- مدل با build\_xgb\_model(random\_state=42) ساخته و fit می‌شود و روی هر دو مجموعه Train و Val مانیتور می‌گردد.
- ارزیابی روی Validation و Test با آستانه پیش‌فرض predict انجام می‌شود (یعنی threshold استاندارد).
- خروجی‌ها:

○ فایل متریک‌ها در METRICS\_DIR/phase2/xgb\_v0\_metrics.json

○ نمودارها در CHARTS\_DIR/phase2/xgb\_v0/

## ۲,۶- آموزش مدل نسخه بهبود یافته (xgb\_v1 (FE + Tuned + Threshold :

در نسخه‌ی دوم، ۳ تغییر کلیدی اعمال شده است:

### الف) افزودن Feature Engineering

قبل از تقسیم Train/Val و همچنین روی Test، ویژگی‌های جدید اضافه می‌شوند.

### ب) استفاده از مدل tuned

به جای مدل پایه، از `xgb_v1_fe_tuned()` استفاده شده که نشان‌دهنده‌ی تنظیمات بهینه‌تر نسبت به نسخه v0 است.

### ج) کاهش آستانه تصمیم برای افزایش Recall

برای مجموعه Test به جای `predict`، ابتدا احتمال کلاس Attack محاسبه شده و سپس با یک آستانه پایین‌تر تصمیم‌گیری انجام می‌شود:

• `y_pred_test = (y_proba_test > 0.35).astype(int)`

این کار معمولاً Recall را افزایش می‌دهد (شکار حمله‌ها بیشتر می‌شود) اما ممکن است Precision را کاهش دهد (False Positive بیشتر).

خروجی‌های این نسخه نیز مشابه v0 ذخیره می‌شوند:

• `xgb_v1_metrics.json`

• نمودارها در `CHARTS_DIR/phase2/xgb_v1/`

### ۳ خروجی‌های عددی و جدول مقایسه مدل‌ها

۳،۱- متریک‌های مدل‌ها روی مجموعه Test

برای مقایسه‌ی منصفانه، معیارهای زیر روی Test گزارش شده‌اند: Accuracy، Precision، Recall، F1 و ROC-AUC. محاسبه‌ی متریک‌ها مطابق تابع `compute_classification_metrics` انجام شده است.

جدول مقایسه (Test)

| مدل                                           | Accuracy | Precision | Recall | F1     | ROC-AUC |
|-----------------------------------------------|----------|-----------|--------|--------|---------|
| Logistic Regression (C=10)                    | 0.7269   | 0.7150    | 0.6471 | 0.6794 | 0.7866  |
| XGBoost (xgb_v0)                              | 0.8842   | 0.9938    | 0.7456 | 0.8520 | 0.8689  |
| XGBoost (xgb_v1: FE + tuned + threshold=0.35) | 0.7731   | 0.7211    | 0.8030 | 0.7598 | 0.8714  |

۳،۲- تحلیل نتایج و دلیل تفاوت‌ها

(۱) بهترین Precision مربوط به xgb\_v0 است.

در xgb\_v0 پرسپژن حدود 0.994 گزارش شده، یعنی وقتی مدل «Attack» اعلام می‌کند، تقریباً همیشه درست است؛ اما Recall آن 0.746 است، یعنی بخشی از حمله‌ها را از دست می‌دهد.

(۲) نسخه xgb\_v1 با کاهش آستانه، Recall را بالا برده است.

در xgb\_v1 روی Test، Recall به 0.803 رسیده که نشان می‌دهد مدل حمله‌های بیشتری را کشف می‌کند. این نتیجه با تصمیم‌گیری با آستانه‌ی 0.35 (به جای 0.5) سازگار است. در عوض Precision کاهش پیدا کرده (0.721) که نشان‌دهنده‌ی افزایش False Positive است.

(۳) ROC-AUC دو مدل XGB تقریباً نزدیک است.

ROC-AUC برای xgb\_v0 حدود 0.869 و برای xgb\_v1 حدود 0.871 است؛ یعنی از نظر “توان تفکیک‌پذیری کلی” هر دو خوب‌اند، اما تفاوت اصلی در نقطه تصمیم (threshold) و Trade-off بین Precision/Recall است.

(۴) Logistic Regression ضعیف‌تر از XGB عمل کرده است.

هم در Accuracy و هم در ROC-AUC پایین‌تر است. (AUC≈0.787) این مدل به عنوان یک baseline ساده قابل گزارش است.

۳،۳- Confusion Matrix (روی Test) و برداشت سریع

برای کامل بودن تحلیل خطا، می‌توان از ماتریس درهم‌ریختگی هم جمع‌بندی کرد:

- **Logistic Regression:**

→  $TN=835, FP=220, FN=301, TP=552$  هم FP و هم FN نسبتاً زیاد است.

- **xgb\_v0:**

→  $TN=1051, FP=4, FN=217, TP=636$  بسیار کم (Precision عالی)، اما FN هنوز قابل توجه است (Recall متوسط).

- **xgb\_v1:**

→  $TN=790, FP=265, FN=168, TP=685$  کمتر شده (Recall بهتر)، ولی FP زیاد شده (Precision افت کرده).

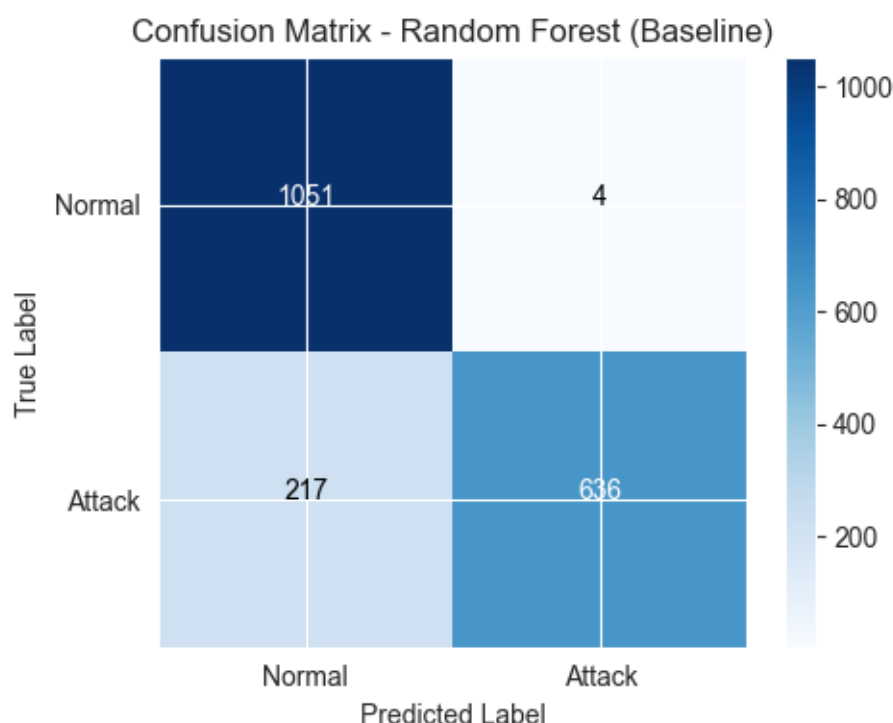


## ۴) نمودارها و شکل‌ها

در این مرحله، برای ارائه‌ی بصری عملکرد مدل‌ها و مقایسه‌ی آن‌ها، مجموعه‌ای از نمودارهای استاندارد طبقه‌بندی رسم شده است. این نمودارها شامل **ROC Curve**، **Confusion Matrix**، نمودارهای مقایسه‌ی متریک‌ها و همچنین **Training Curve (logloss)** برای بررسی روند یادگیری مدل هستند.

### ۴,۱- Confusion Matrix

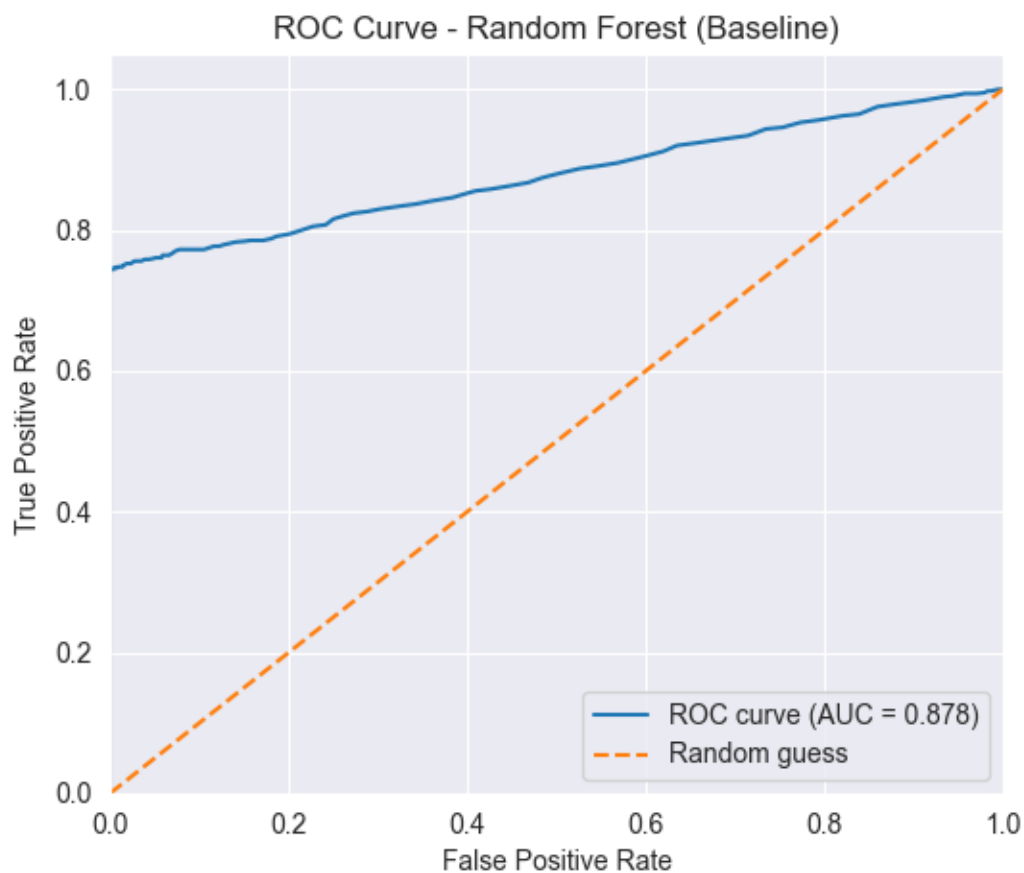
شکل ۴-۱: Confusion Matrix – Baseline (Random Forest / یا مدل پایه پروژه)



این شکل تعداد پیش‌بینی‌های درست و غلط را برای دو کلاس **Normal** و **Attack** نمایش می‌دهد. طبق ماتریس، مقدار **False Positive** بسیار کم ( $FP=4$ ) است و در مقابل **False Negative** ( $FN=217$ ) نشان می‌دهد بخشی از حمله‌ها از دست می‌روند؛ بنابراین مدل پایه از نظر **Precision** بسیار خوب است اما **Recall** محدودتر دارد.

### ۴,۲- ROC Curve

شکل ۴-۲: ROC Curve – Baseline (Random Forest / مدل پایه)



“ROC curve of the baseline model on the test data along with the AUC value; comparison with the random guess line”.

این نمودار رابطه‌ی **True Positive Rate** و **False Positive Rate** را در آستانه‌های مختلف نشان می‌دهد. مقدار **AUC** معیار کلی قدرت جداسازی کلاس‌هاست؛ هرچه AUC بزرگ‌تر باشد مدل بهتر می‌تواند Attack را از Normal جدا کند (بدون وابستگی مستقیم به threshold).

### ۴,۳- نمودارهای مقایسه‌ی متریک‌ها بین مدل‌ها

برای مقایسه‌ی سریع مدل‌ها، نمودار میله‌ای متریک‌های زیر رسم شده است:

#### • شکل ۴-۳: **Accuracy Comparison Between Models**

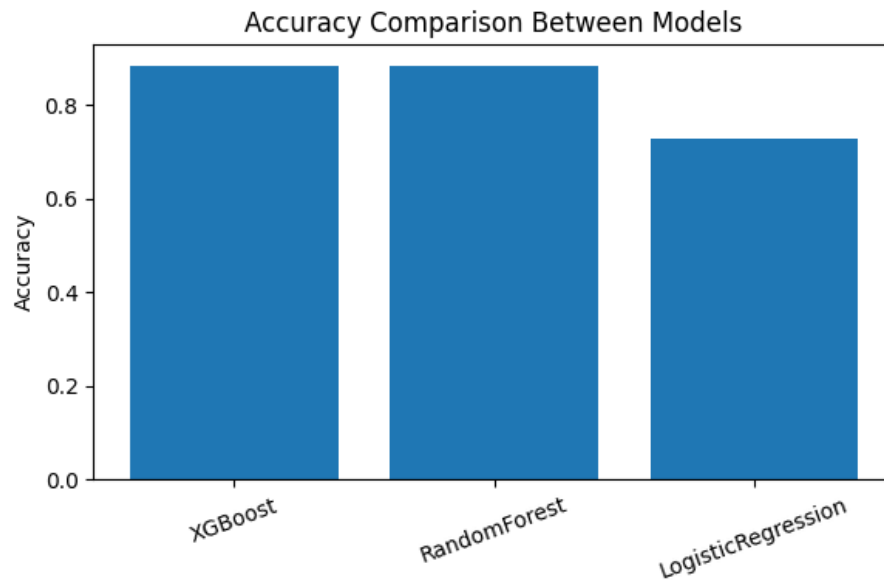
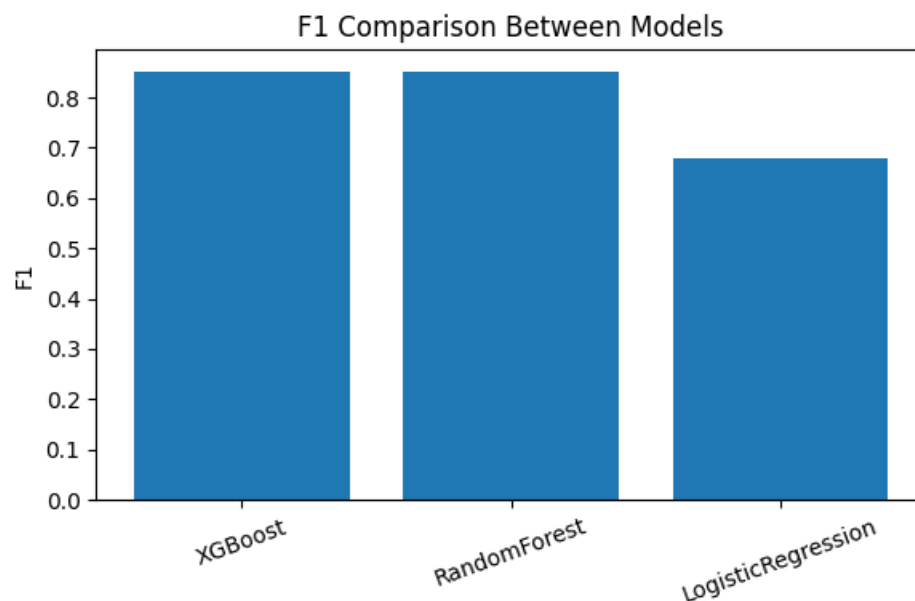


Figure 1 “Comparison of performance metrics (Accuracy) among Logistic Regression, the baseline model (Random Forest), and XGBoost.”

معیار Accuracy دقت کلی مدل‌ها را نشان می‌دهد. در پروژه‌ی تشخیص نفوذ، Accuracy به‌تنهایی معیار کافی نیست، چون هزینه‌ی از دست دادن حمله‌ها (FN) بالاست.

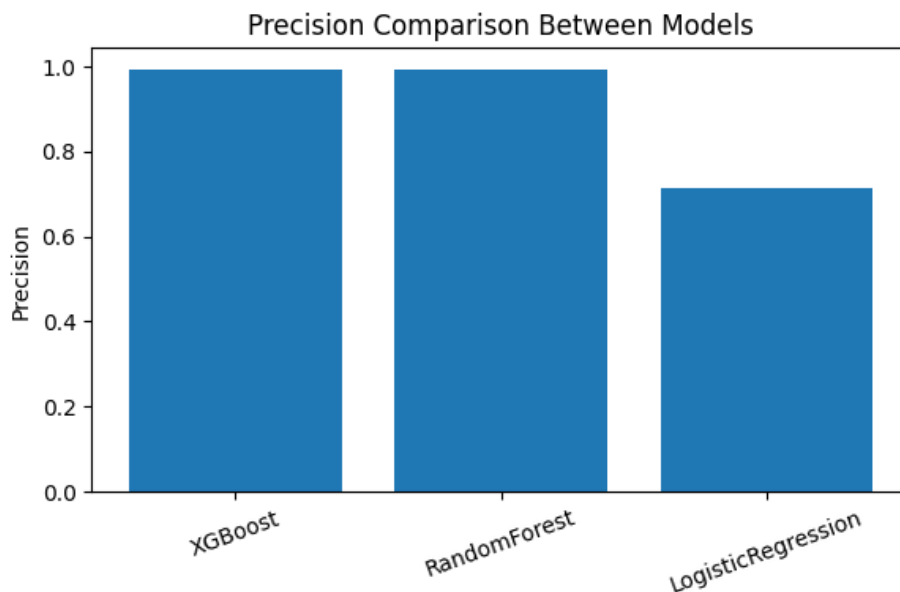
#### شکل ۴-۱: F1 Comparison Between Models :



“Comparison of performance metrics (F1) among Logistic Regression, the baseline model (Random Forest), and XGBoost.”

F1 تعادل بین Precision و Recall را نشان می‌دهد و برای مقایسه‌ی مدل‌ها در مسائل دودویی کاربردی است.

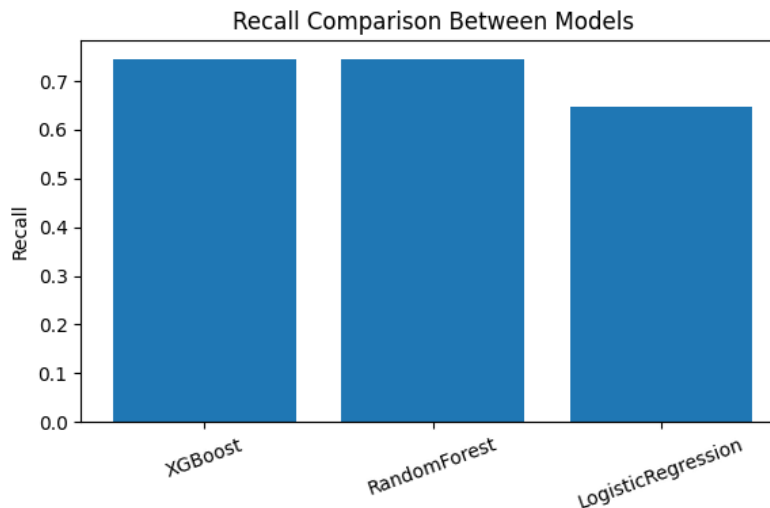
#### شکل ۵-۴: Precision Comparison Between Models



"Comparison of performance metrics ( Precision) among Logistic Regression, the baseline model (Random Forest), and XGBoost."

نشان می‌دهد وقتی مدل «Attack» اعلام می‌کند، چقدر قابل اعتماد است (کم بودن FP).

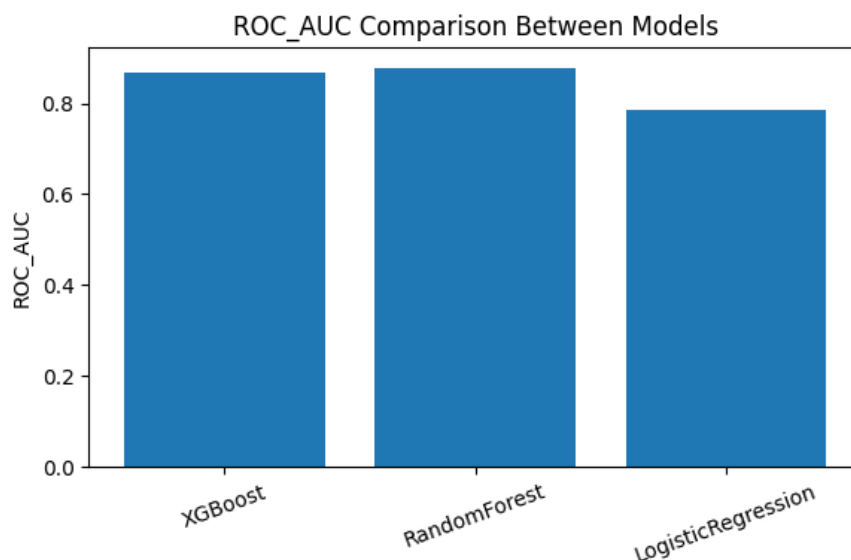
#### شکل ۶-۴: Recall Comparison Between Models



"Comparison of performance metrics ( Recall) among Logistic Regression, the baseline model (Random Forest), and XGBoost."

- مهم‌ترین نمودار برای مسئله‌ی ما، چون Recall نشان می‌دهد چه درصدی از حمله‌ها کشف شده‌اند (کم بودن FN).

- **شکل ۴-۷: ROC AUC Comparison Between Models :**



- *“Comparison of performance metrics (ROC-AUC) among Logistic Regression, the baseline model (Random Forest), and XGBoost.”*

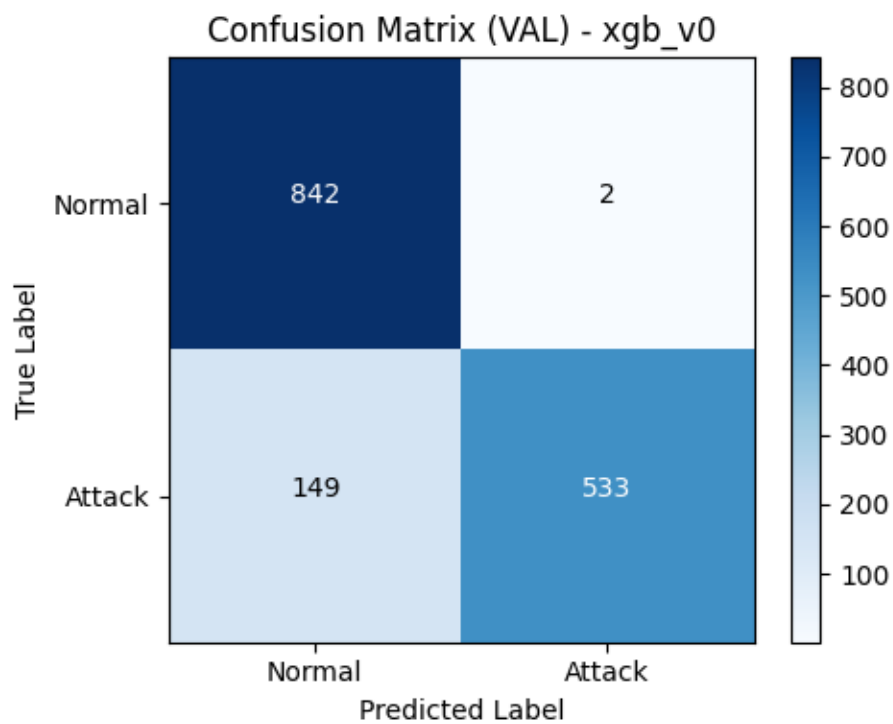
- مقایسه‌ی توان جداسازی کلی مدل‌ها مستقل از آستانه.

در مسئله‌ی Intrusion Detection ، **Recall** اهمیت ویژه دارد زیرا از دست دادن حمله (FN) هزینه‌ی بالایی دارد. به همین دلیل، حتی اگر Precision اندکی افت کند، افزایش Recall می‌تواند مطلوب‌تر باشد.

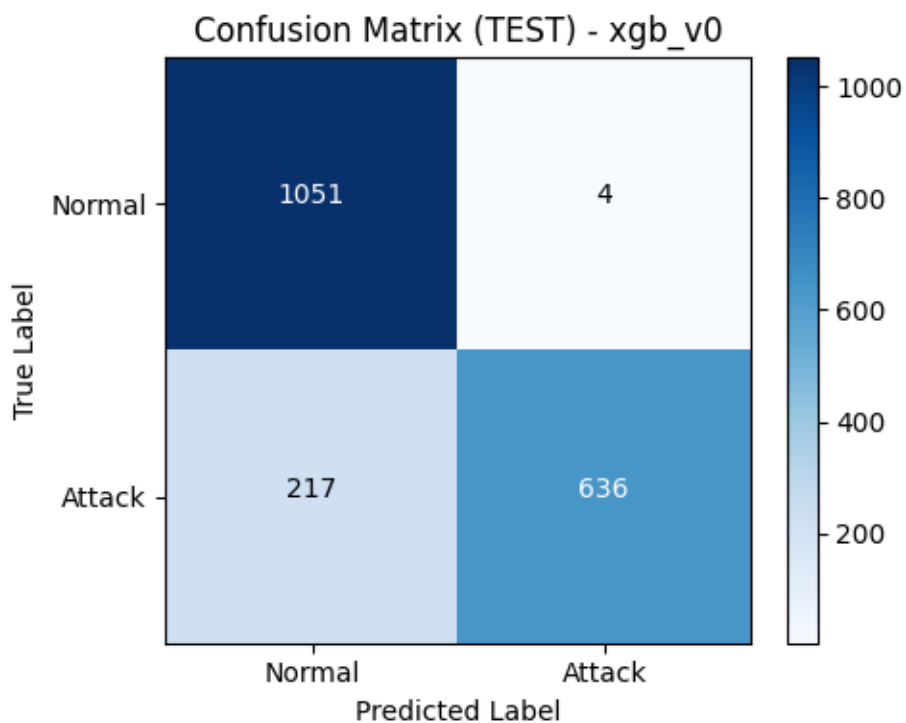
۴,۴- نمودارهای اختصاصی XGBoost v0 و XGBoost v1 (Train/Val/Test)

الف) Confusion Matrix ( VAL و TEST )

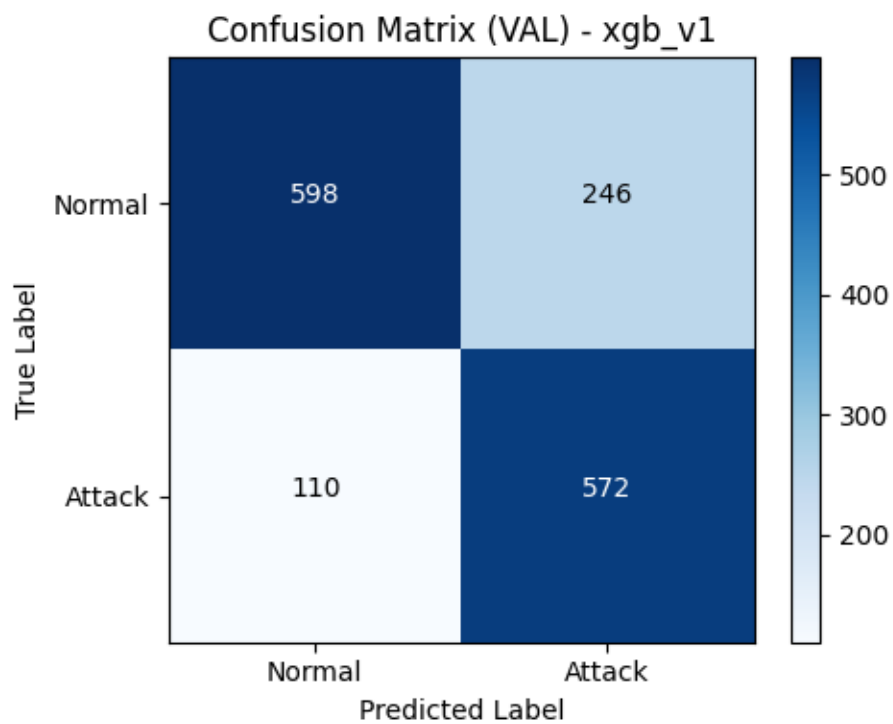
- **شکل ۴-۸: Confusion Matrix (VAL) – xgb\_v0 :**



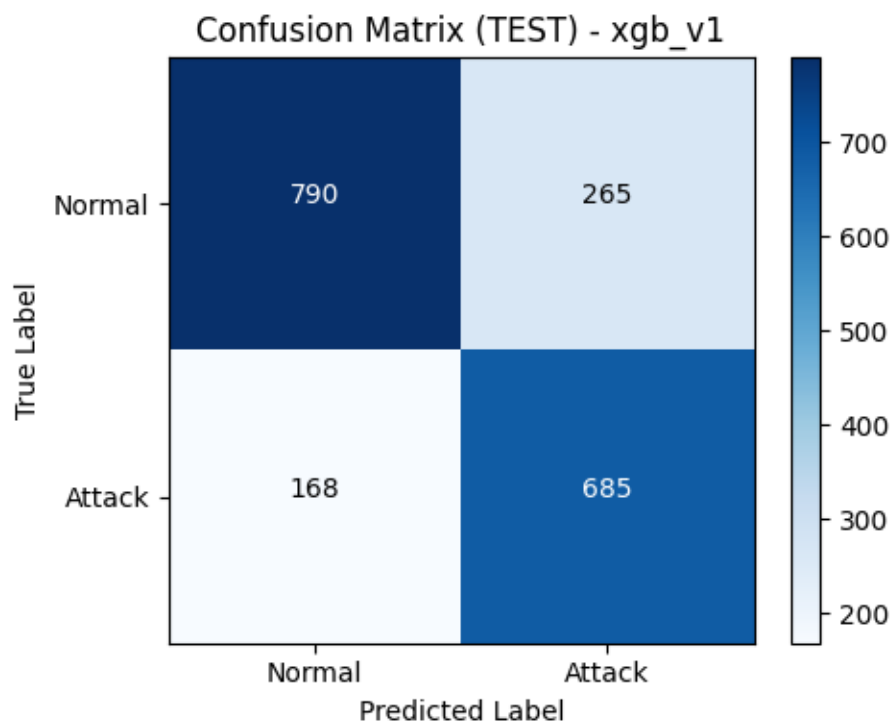
• شکل ٤-٩: Confusion Matrix (TEST) – xgb\_v0



• شکل ٤-١٠: Confusion Matrix (VAL) – xgb\_v1



• شکل ٤-١ : Confusion Matrix (TEST) – xgb\_v1

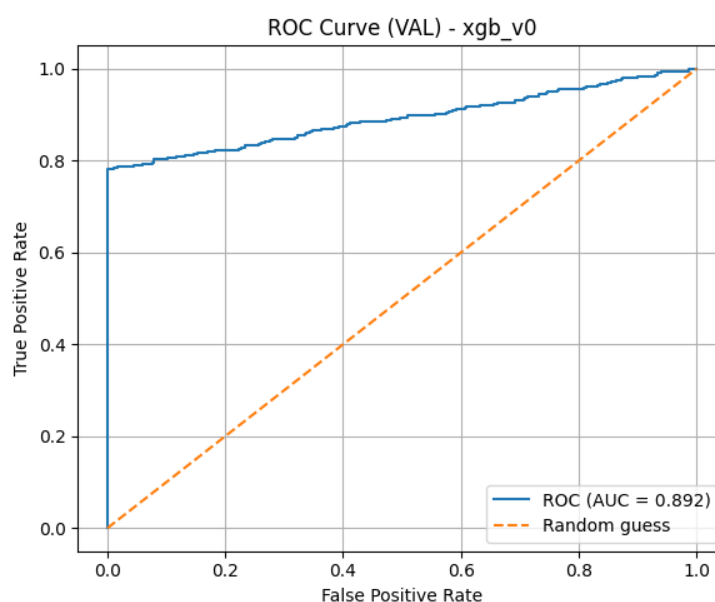


نکته قابل توجه:

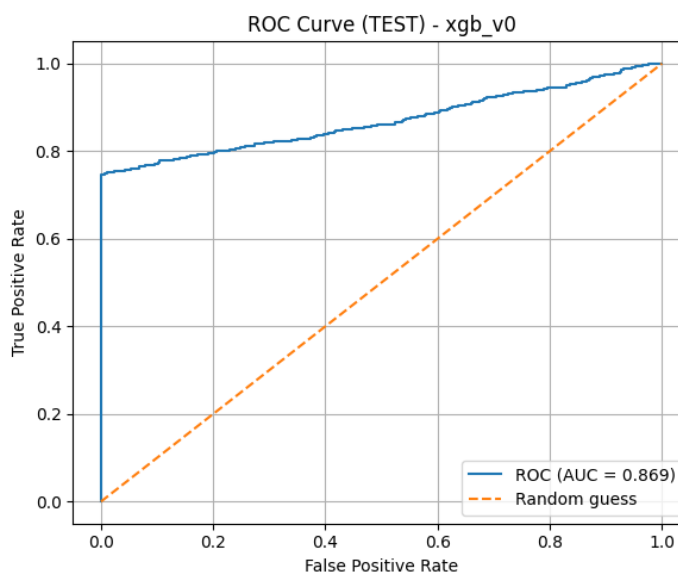
نسخه‌ی v1 با تغییرات ( Feature Engineering + tuned + threshold ) معمولاً باعث کاهش FN و در نتیجه افزایش Recall می‌شود، اما در عوض ممکن است FP افزایش یابد و Precision کاهش پیدا کند. این دقیقاً همان trade-off بین Precision و Recall است که در تشخیص نفوذ مهم است.

ب) ROC Curve ( VAL و TEST )

• شکل ۴-۱۲ – ROC Curve ( VAL ) – xgb\_v0

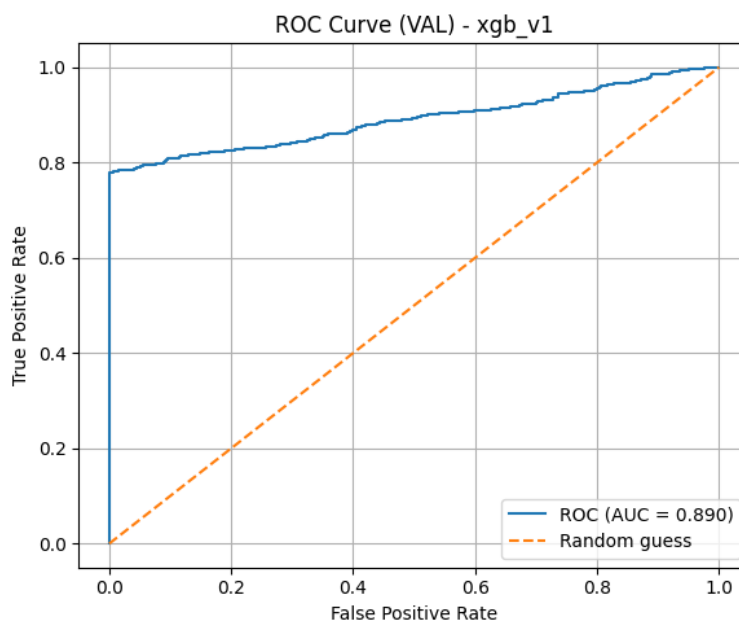


• شکل ۴-۱۳ – ROC Curve ( TEST ) – xgb\_v0

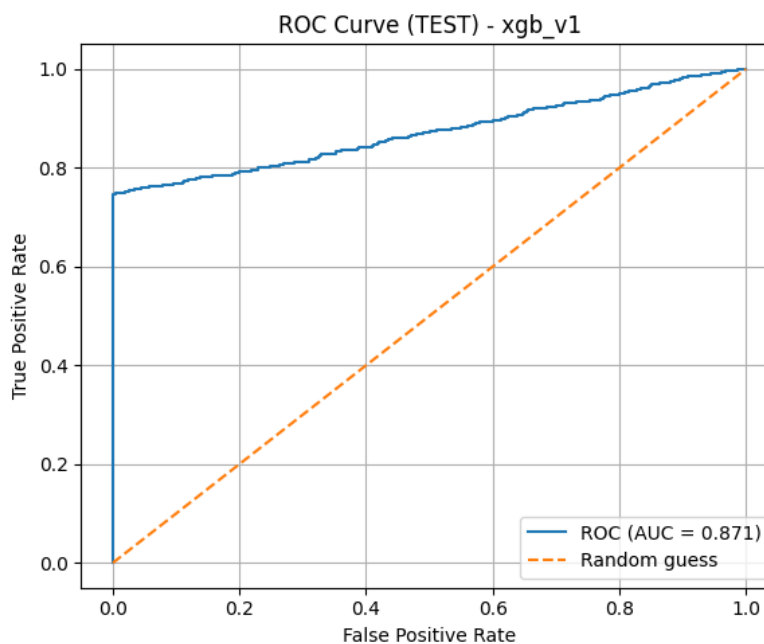




• شکل ۴-۱۴ – ROC Curve (VAL) – xgb\_v1 :



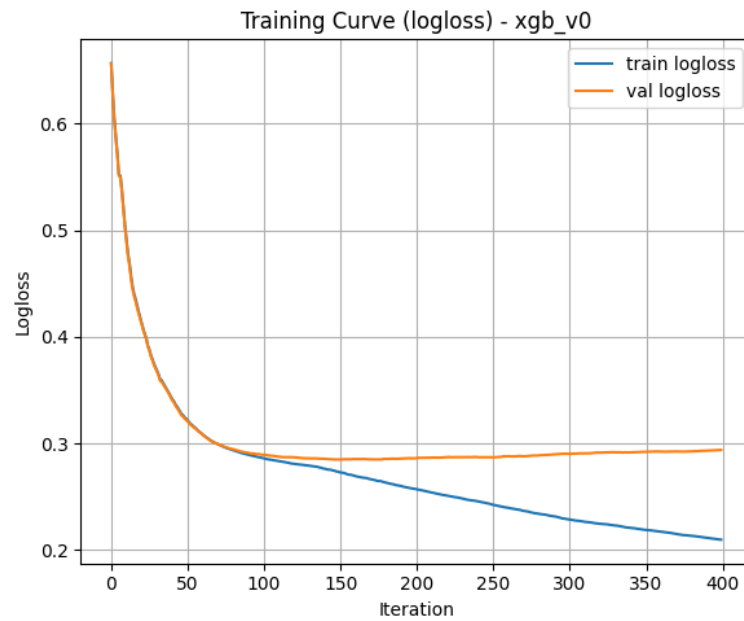
• شکل ۴-۱۵ – ROC Curve (TEST) – xgb\_v1 :



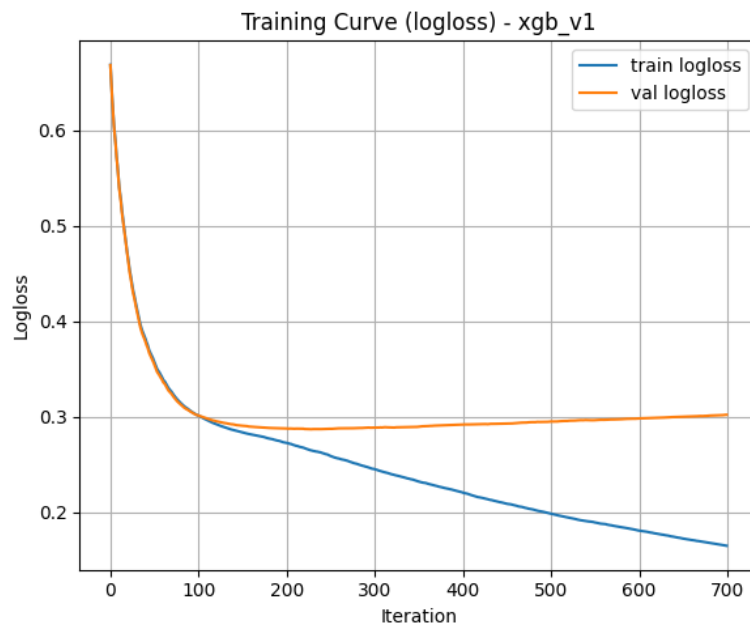
نزدیکی AUC در v0 و v1 نشان می‌دهد قدرت جداسازی کلی مشابه است، اما تفاوت اصلی در آستانه تصمیم و مدیریت trade-off داده است.

(ج) Training Curve (logloss)

• شکل ۴-۱۶ - xgb\_v0 : Training Curve (logloss)



• شکل ۴-۱۷ - xgb\_v1 : Training Curve (logloss)



این نمودار روند کاهش logloss در Train و Val را در طول iteration ها نشان می‌دهد و برای تشخیص **overfitting/underfitting** کاربرد دارد (مثلاً اگر فاصله‌ی Train و Val زیاد شود، نشانه‌ی **overfitting** است).

## ۵) دمو (Inference / Demo)

برای نشان دادن قابلیت استفاده‌ی مدل در حالت واقعی (بدون آموزش مجدد)، یک دمو برای پیش‌بینی نفوذ پیاده‌سازی شد. در این دمو، چند نمونه‌ی ورودی از فایل `demo_samples.csv` خوانده می‌شود و پس از انجام همان مراحل آماده‌سازی ویژگی‌ها، مدل برای هر نمونه:

۱. احتمال حمله (`attack_probability`)

۲. برچسب نهایی حمله/نرمال (`attack_predicted`) را تولید می‌کند.

نحوه اجرای دمو:

دمو از طریق اجرای ماژول زیر در محیط پروژه اجرا می‌شود:

```
python -m src.inference.demo_predicted
```

ورودی دمو یک فایل CSV شامل چند ردیف نمونه است (برای تست سریع)، مانند:

- `demo_samples.csv`

این فایل شامل ویژگی‌های مربوط به `session/traffic` بوده و در صورت نیاز، ویژگی‌های مهندسی شده نیز (مانند `ip_attack_risk`) در فرآیند `inference` ساخته/استفاده می‌شوند.

پس از اجرا، خروجی به صورت یک جدول (`DataFrame`) چاپ می‌شود که علاوه بر ویژگی‌ها، دو ستون کلیدی دارد:

- `attack_probability`: احتمال تعلق نمونه به کلاس `Attack`

- `attack_predicted`: خروجی نهایی مدل ( `Attack=1` ، `Normal=0` )

نمونه خروجی دمو نشان می‌دهد که مدل برای هر ردیف، احتمال حمله را محاسبه کرده و سپس تصمیم نهایی را بر اساس آستانه‌ی تصمیم (`threshold`) تولید می‌کند.

نکته: وجود ستون `attack_probability` باعث می‌شود بتوانیم بسته به نیاز سیستم `IDS`، آستانه را تغییر دهیم؛ مثلاً در سناریوهای حساس، آستانه را کاهش می‌دهیم تا `Recall` بالاتر شود و حمله‌های کمتری از دست بروند.

```
(.venv) PS C:\Users\Asus\PycharmProjects\AIProject> python -m src.inference.demo_predicted
network_packet_size login_attempts session_duration ... ip_attack_risk attack_probability attack_predicted
0 -0.097015 -0.5 1.151995 ... 0.047654 0.096709 0
1 1.141791 0.0 0.057433 ... 0.724850 0.976248 1
2 -0.167910 0.5 -0.195146 ... 0.406332 0.076832 0
[3 rows x 23 columns]
```

اسکرین‌شات اجرای دمو (اجرای دستور و نمایش خروجی ۳ ردیف نمونه و ستون‌های `attack_probability` و `attack_predicted`) درج شده است تا قابلیت اجرای `end-to-end` مدل (از ورودی CSV تا خروجی نهایی) قابل مشاهده و قابل ارائه باشد.

## ۶) مدل ذخیره‌شده

برای اینکه سیستم IDS بتواند در حالت عملیاتی (Deployment/Inference) بدون نیاز به آموزش مجدد اجرا شود، مدل نهایی آموزش‌دیده به صورت یک فایل artifact ذخیره شده است. این کار باعث می‌شود در زمان پیش‌بینی فقط مدل load شود و برای داده‌های جدید، خروجی احتمال حمله و برچسب نهایی تولید گردد.

### فرمت و محل ذخیره‌سازی مدل

مدل نهایی با نام زیر ذخیره شده است:

- `xgb_v1.joblib`

فرمت `joblib` برای ذخیره‌سازی مدل‌های مبتنی بر `scikit-learn` (و مدل‌هایی که API مشابه دارند مانند `XGBoost sklearn wrapper`) رایج است و امکان بارگذاری سریع مدل را فراهم می‌کند.

### نحوه بارگذاری مدل و استفاده در پیش‌بینی

برای اطمینان از صحت ذخیره‌سازی و امکان استفاده‌ی مجدد، یک اسکریپت `inference` نوشته شده است که:

۱. داده‌های `preprocessed` را بارگذاری می‌کند،
۲. روی داده‌ی تست **Feature Engineering** امنیتی را اعمال می‌کند،
۳. مدل ذخیره‌شده را از مسیر `load MODELS_DIR` می‌کند،
۴. احتمال کلاس `Attack` را با `predict_proba` محاسبه کرده و سپس با `threshold` (پیش‌فرض ۰٫۵) برچسب نهایی را تولید می‌کند،
۵. و در پایان یک خروجی `sanity-check` چاپ می‌کند (تعداد پیش‌بینی‌های ۰ و ۱).
- ۶.

### نکته مهم درباره سازگاری ورودی مدل:

از آن‌جا که مدل `xgb_v1` با `Feature Engineering` آموزش داده شده است، در مرحله‌ی `inference` نیز باید همان ویژگی‌های مهندسی‌شده (مانند `gip_attack_risk` و سایر فیچرهای امنیتی) به ورودی اضافه شوند تا ورودی مدل با زمان آموزش سازگار بماند.

### آستانه تصمیم (Threshold) در زمان استفاده از مدل ذخیره‌شده

در اسکریپت بررسی مدل ذخیره‌شده، `threshold` پیش‌فرض ۰٫۵ استفاده شده است:

- `y_pred = (y_proba >= 0.5).astype(int)`

با توجه به ماهیت IDS، امکان تغییر `threshold` وجود دارد؛ برای مثال در سناریوهایی که هدف افزایش کشف حمله‌ها است، می‌توان `threshold` را کاهش داد تا **Recall** افزایش پیدا کند (با احتمال افزایش `False Positive`).

## ۷ نتیجه

در این پروژه یک سیستم تشخیص نفوذ (IDS) به صورت یک مسئله طبقه‌بندی دودویی پیاده‌سازی شد که هدف آن تشخیص کلاس Attack در کنار کلاس Normal است. در فاز دوم، تمرکز اصلی روی بهبود کیفیت مدل و نزدیک‌تر کردن پروژه به حالت قابل استفاده (inference/demo) بود.

### - جمع‌بندی عملکرد مدل‌ها و انتخاب مدل نهایی

در مقایسه‌ی مدل‌های آزمایش‌شده، مدل‌های XGBoost نسبت به Logistic Regression عملکرد بهتری ارائه کردند. نسخه‌ی نهایی انتخاب‌شده، XGBoost نسخه v1 است که با Feature Engineering و تنظیمات بهبود یافته، توانست عملکرد بهتری مخصوصاً از نظر کشف حمله‌ها (Recall) ارائه دهد. بر اساس خروجی‌های ارزیابی، مدل نهایی دارای توان جداسازی کلی مناسب بوده و مقدار AUC حدود ۰.۸۷، را روی داده‌ی Test نشان می‌دهد که بیانگر قدرت تفکیک خوب بین حمله و رفتار عادی است. شکل (ROC – Test)

### - تفسیر ROC و اهمیت آن برای IDS

منحنی ROC مربوط به Test نشان می‌دهد که مدل در بازه‌های مختلف آستانه تصمیم‌گیری، Trade-off مناسبی بین True Positive Rate و False Positive Rate برقرار می‌کند. از آن‌جا که در IDS هزینه‌ی از دست دادن حمله‌ها (False Negative) بالاست، مدل نهایی به گونه‌ای انتخاب شده که در کنار AUC مناسب، قابلیت تنظیم آستانه برای افزایش Recall را نیز داشته باشد.

### - خروجی‌پذیری و قابلیت استفاده عملی (Demo / Inference)

برای اثبات کاربردی بودن مدل، یک دمو برای inference پیاده‌سازی شد.

در دمو، چند نمونه‌ی ورودی خوانده می‌شود و خروجی به شکل یک جدول نمایش داده می‌شود که شامل:

- attack\_probability (احتمال حمله)

- attack\_predicted (برچسب نهایی ۱/۰)

است. خروجی ثبت‌شده نشان می‌دهد مدل برای هر نمونه احتمال حمله را محاسبه کرده و سپس برچسب نهایی را تولید می‌کند؛ بنابراین پروژه از حالت صرفاً آموزشی خارج شده و قابلیت استفاده‌ی end-to-end برای پیش‌بینی را دارد.

### - ذخیره‌سازی مدل و تکرارپذیری

مدل نهایی برای استفاده‌ی مجدد به صورت artifact ذخیره شده است (xgb\_v1.joblib) تا در سناریوهای عملیاتی، بدون نیاز به آموزش مجدد قابل بارگذاری و استفاده باشد. این کار باعث افزایش تکرارپذیری و ارائه‌پذیری پروژه می‌شود.

### - نتیجه نهایی

در مجموع، پروژه موفق شد یک IDS مبتنی بر یادگیری ماشین ارائه دهد که:

- از نظر تفکیک Attack/Normal عملکرد مناسبی دارد (AUC روی Test  $\approx 0.87$ )،
- امکان استفاده‌ی عملی از طریق دمو و تولید خروجی احتمال/برچسب را فراهم می‌کند،

- و مدل نهایی را به صورت فایل ذخیره شده جهت inference مجدد ارائه می دهد.

این موارد نشان می دهد مدل نهایی علاوه بر عملکرد قابل قبول، از نظر مهندسی و ارائه نیز برای تحویل پروژه و نمایش دمو آماده است.